Hands-On Report

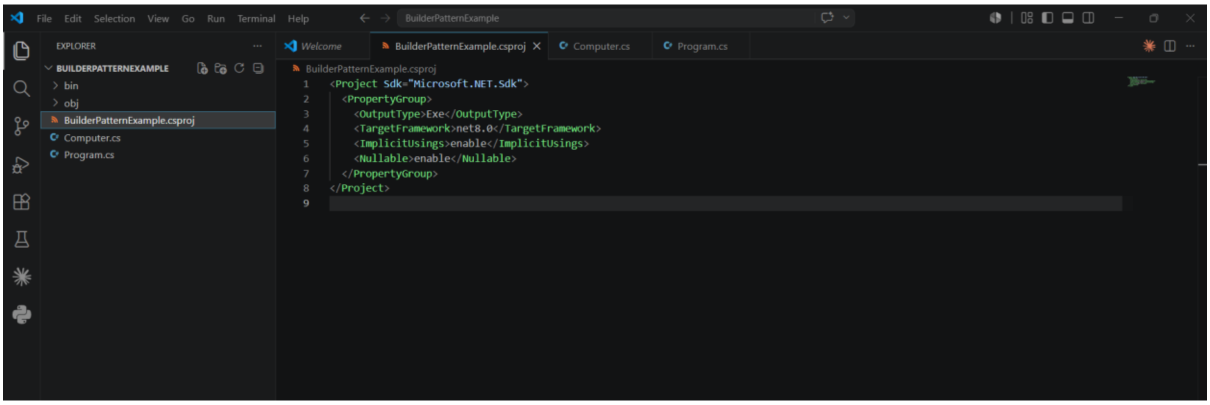
Exercise 3: Implementing the Builder Pattern

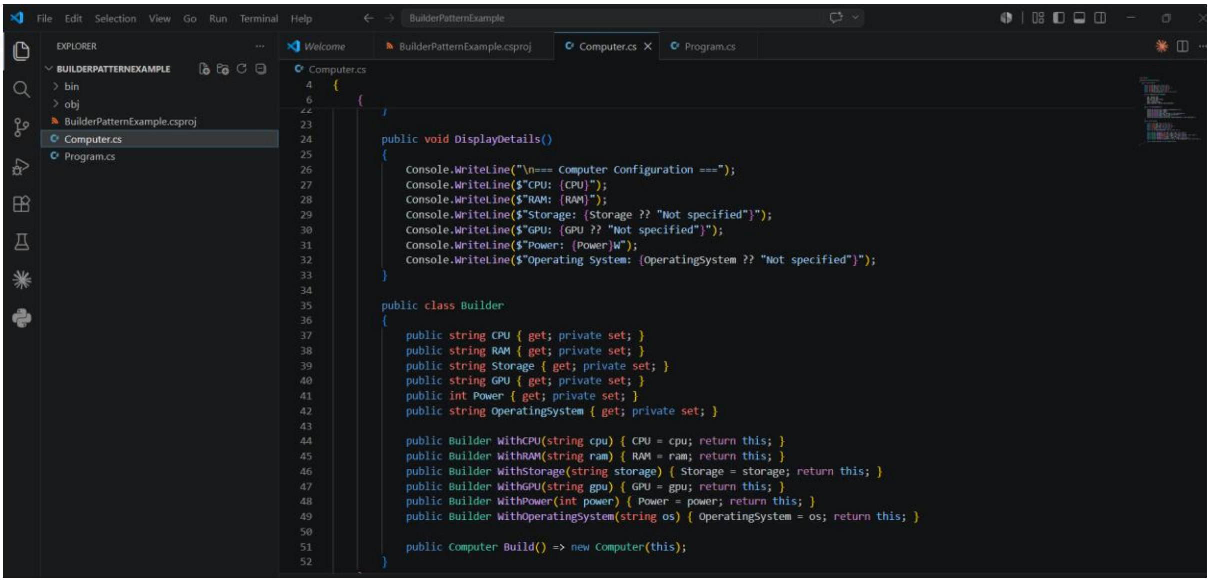
In this exercise, I implemented and tested the Builder Pattern in a .NET application by developing a project named **BuilderPatternExample**. I created a **Computer** class containing properties such as CPU, RAM, and Storage, along with a **ComputerBuilder** class that allows these properties to be configured step by step before creating the final object.

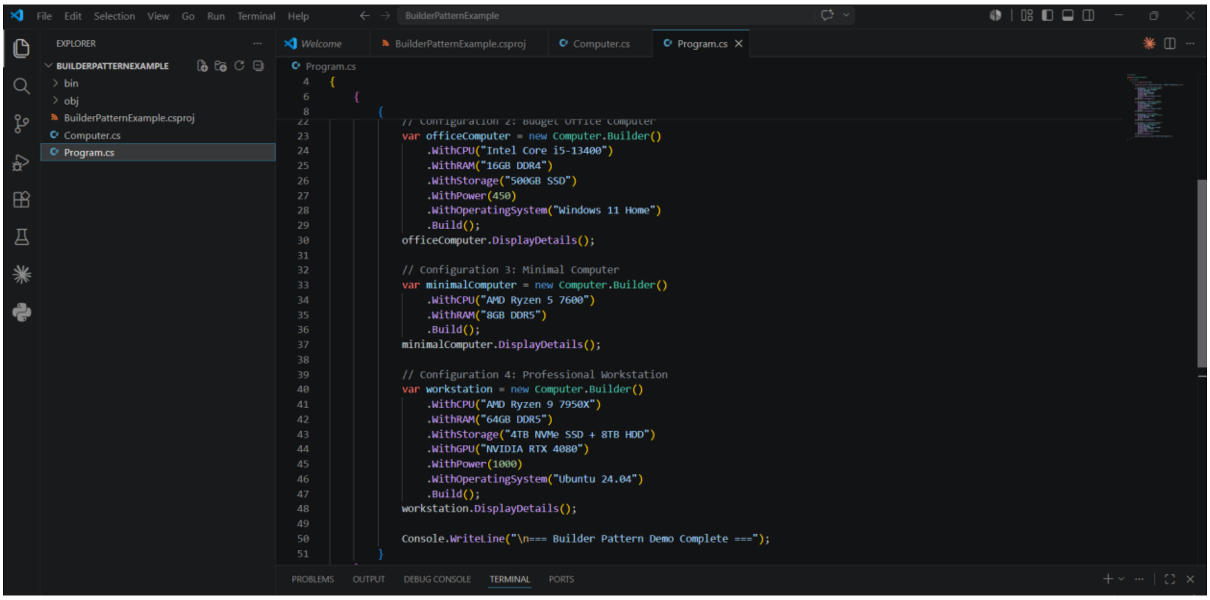
A private constructor was used to ensure that objects are instantiated only through the builder, resulting in a structured and controlled object creation process. After running the application, I tested multiple computer configurations to confirm that the builder generated the required objects correctly.

This hands-on activity helped me understand how the Builder Pattern simplifies the construction of complex objects, improves code organization, enhances flexibility when handling optional parameters, and increases the maintainability of .NET applications.

Coding:


File Edit Selection View Go Run Terminal Help
BuilderPatternExample
EXPLORER
BUILDERPATTERNEXAMPLE
bin
obj
BuilderPatternExample.csproj
Computer.cs
Program.cs
Welcome
BuilderPatternExample.csproj
Computer.cs
Program.cs
BuilderPatternExample.csproj
<Project Sdk="Microsoft.NET.Sdk">
<PropertyGroup>
<OutputType>Exe</OutputType>
<TargetFramework>net8.0</TargetFramework>
<ImplicitUsings>enable</ImplicitUsings>
<Nullable>enable</Nullable>
</PropertyGroup>
</Project>


File Edit Selection View Go Run Terminal Help
BuilderPatternExample
EXPLORER
BUILDERPATTERNEXAMPLE
bin
obj
BuilderPatternExample.csproj
Computer.cs
Program.cs
Welcome
BuilderPatternExample.csproj
Computer.cs
Program.cs
Computer.cs
public void DisplayDetails()
{
Console.WriteLine("\n=== Computer Configuration ===");
Console.WriteLine($"CPU: {CPU}");
Console.WriteLine($"RAM: {RAM}");
Console.WriteLine($"Storage: {Storage ?? "Not specified"}");
Console.WriteLine($"GPU: {GPU ?? "Not specified"}");
Console.WriteLine($"Power: {Power}W");
Console.WriteLine($"Operating System: {OperatingSystem ?? "Not specified"}");
}
public class Builder
{
public string CPU { get; private set; }
public string RAM { get; private set; }
public string Storage { get; private set; }
public string GPU { get; private set; }
public int Power { get; private set; }
public string OperatingSystem { get; private set; }
public Builder WithCPU(string cpu) { CPU = cpu; return this; }
public Builder WithRAM(string ram) { RAM = ram; return this; }
public Builder WithStorage(string storage) { Storage = storage; return this; }
public Builder WithGPU(string gpu) { GPU = gpu; return this; }
public Builder WithPower(int power) { Power = power; return this; }
public Builder WithOperatingSystem(string os) { OperatingSystem = os; return this; }
public Computer Build() => new Computer(this);
}


File Edit Selection View Go Run Terminal Help
BuilderPatternExample
EXPLORER
BUILDERPATTERNEXAMPLE
bin
obj
BuilderPatternExample.csproj
Computer.cs
Program.cs
Welcome
BuilderPatternExample.csproj
Computer.cs
Program.cs
Program.cs
{
{
{
// Configuration 2: Budget Office Computer
var officeComputer = new Computer.Builder()
.WithCPU("Intel Core i5-13400")
.WithRAM("16GB DDR4")
.WithStorage("500GB SSD")
.WithPower(450)
.WithOperatingSystem("Windows 11 Home")
.Build();
officeComputer.DisplayDetails();
// Configuration 3: Minimal Computer
var minimalComputer = new Computer.Builder()
.WithCPU("AMD Ryzen 5 7600")
.WithRAM("8GB DDR5")
.Build();
minimalComputer.DisplayDetails();
// Configuration 4: Professional Workstation
var workstation = new Computer.Builder()
.WithCPU("AMD Ryzen 9 7950X")
.WithRAM("64GB DDR5")
.WithStorage("4TB NVMe SSD + 8TB HDD")
.WithGPU("NVIDIA RTX 4080")
.WithPower(1000)
.WithOperatingSystem("Ubuntu 24.04")
.Build();
workstation.DisplayDetails();
Console.WriteLine("\n=== Builder Pattern Demo Complete ===");
}
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS